

MTGNP v1.0 — Magic: The Gathering Network Protocol

CSNETWK Machine Problem — implementation of RFC 0001 (MTGNP v1.0).

Demonstration and Gameplay Video

Watch our demonstration and gameplay video here: https://drive.google.com/drive/folders/1nWk_jC-yXjDDTInRrVEfQos8iDjgP5Ge?usp=sharing

Project structure

```
project/
├── server/
│   ├── main.py           # Game Server package – run: python3 -m server.main
│   ├── transport.py      # entry point, socket accept loop, lifecycle (Suñga)
│   ├── heartbeat.py      # framing, PDU codec, ClientConn, dispatch table (Suñga)
│   ├── lobby.py          # PING/PONG (Suñga)
│   ├── phase_engine.py   # PLAYER_READY handling, LOBBY/SETUP/MULLIGAN (Rebudiao)
│   ├── combat.py         # phase/step machine, PHASE_TRANSITION, land drops (Rebudiao)
│   ├── card_catalog.py   # attackers/blockers/damage order/first strike (Barreo)
│   ├── game_state.py     # loads shared JSON catalog; ILLEGAL_DECK validation (Gregorio)
│   ├── state_view.py     # authoritative data model + GameOver/ClientGone (Gregorio)
│   ├── priority.py       # personalized GAME_STATE_UPDATE views, hidden hands (Gregorio)
│   ├── stack.py          # PRIORITY_GRANT/PASS, seq_num tokens, STALE_ACTION (Gregorio)
│   ├── effects.py        # LIFO stack, STACK_PUSH/RESOLVE, SBAs, triggers (Gregorio)
│   └── tests/
│       ├── test_card_catalog.py # card effects, mana payment, INSUFFICIENT_MANA (Gregorio)
│       ├── test_game_state.py   # unit: catalog + deck validation
│       ├── test_priority_stack.py # unit: data model + hidden info
│       ├── test_effects.py      # unit: tokens, LIFO stack, counterspell
│       ├── test_integration_game.py # unit: mana, effects, state-based actions
│       ├── test_integration_combat.py # end-to-end protocol game (10 assertions)
│       └── test_integration_edge.py # combat/effect mechanics (3 assertions)
├── shared/
│   └── cards.json         # edge cases + TRIGGER_ORDER (18 assertions)
├── client/
│   └── main.py            # shared card catalog (out-of-band card data, RFC §1)
├── gui/
│   ├── index.html        # Player Client & rendering – run: python3 -m client.main (Barreo)
│   └── gui_bridge.py      # Browser-based GUI client (open in browser, 2 tabs)
├── gui_bridge.py          # WebSocket-to-TCP bridge for the GUI – run: python gui_bridge.py
├── decks/
│   ├── red_aggro.txt      # sample deck lists (one card instance ID per line)
│   ├── green_stompy.txt  # Red: hasty creatures + burn (40 cards)
│   ├── burn.txt          # Green: big creatures + pump (28 cards)
│   ├── control.txt       # Red: direct damage only (40 cards)
│   ├── white_weenie.txt  # Blue/Black: counters + drain (28 cards)
│   ├── blue_tempo.txt    # White: knights & lions (32 cards)
│   ├── black_aggro.txt   # Blue: counter spells + fliers (32 cards)
│   └── requirements.txt  # Black: knights & drain (20 cards)
├── README.md              # states that only the Python stdlib is required
├── README.pdf             # this document (Markdown source)
└── README.pdf             # this document (submitted deliverable)
```

The server is a Python package: one module per protocol concern, composed into the `Server` class in `server/main.py` via mixins. Method implementations are identical to the reference single-file build; only the file layout differs.

Requirements: **Python 3.10+**, standard library only. No third-party packages.

Running — step by step

Prerequisites: Python 3.10 or newer. No installation of packages is needed (standard library only).

Windows note: the command is `python` (or `py`), not `python3` — substitute it in every command below, e.g. `python server.py --verbose`. If Windows says “*Python was not found*”, install Python from python.org/downloads and tick “**Add python.exe to PATH**” in the installer, then reopen the terminal. Check with `python --version` (Linux/macOS: `python3 --version`).

1. **Unzip the project** and open a terminal in the project folder (the one containing the `server/` package).

Note: You could add `--verbose` for steps 3-5 to enable verbose mode.

3. **Start the server** (Terminal 1). It listens on port 4444 (RFC 5.1):

```
python -m server.main
```

You should see [server] listening on port 4444 . Use `--port <n>` if 4444 is taken (then add `--port <n>` to the clients too).

4. **Start the first client** (Terminal 2):

```
python -m client.main --id player_1 --deck decks/burn.txt
```

If the server is on another machine, add `--host <server-ip>` .

5. **Start the second client** (Terminal 3):

```
python -m client.main --id player_2 --deck decks/control.txt
```

A third connection attempt will be refused by the server (RFC 5.1).

6. **Join the game:** type `ready` in each client and press Enter. When both are ready the server deals hands and the mulligan begins.
7. **Mulligan:** type `keep` to keep your hand, or `mull` to redraw. After N mulligans, keep with `keep <card_1> ... <card_N>` to put N cards on the bottom of your library (London mulligan).
8. **Play:** when you see `>>> You have priority` , you may act:
 - `play mountain_003` — play a land (your main phase only, 1/turn)
 - `cast lightning_bolt_001 player_2` — cast a spell (mana is paid automatically from your untapped lands)
 - `pass` — pass priority
 - `attack goblin_guide_001` — declare attackers (in DECLARE_ATTACKERS)
 - `block wall_of_stone_001:goblin_guide_001` — declare blockers
 - `order <attacker> <blocker1> <blocker2>` — damage order (multi-blocks)
 - `discard <card>` — discard to 7 at cleanup; `yes / no` — answer an optional trigger; `hand / state` — re-print; `concede` — give up; `help` — full list
9. **Game over:** the winner and reason are announced; both clients stay connected and can type `ready` to start a new game (RFC 6.6).

Running the GUI (browser client)

The project includes a browser-based GUI as an alternative to the terminal client. It connects to the game server through a WebSocket-to-TCP bridge (`gui_bridge.py`).

Extra prerequisite: the `websockets` Python package (the only non-stdlib dependency in the project).

```
pip install websockets
```

1. **Start the game server** (Terminal 1) — same as above:

```
python -m server.main
```

2. **Start the WebSocket bridge** (Terminal 2). It connects to the game server on TCP port 4444 and exposes a WebSocket on port 8765:

```
python gui_bridge.py
```

You should see `WebSocket bridge starting on ws://0.0.0.0:8765` . Optional flags:

- `--host <ip>` — game server IP (default `127.0.0.1`)
 - `--port <n>` — game server TCP port (default `4444`)
 - `--ws-port <n>` — WebSocket port for browsers (default `8765`)
 - `-v / --verbose` — log every PDU forwarded through the bridge
3. **Open the GUI** — open `gui/index.html` in **two browser tabs** (one per player). Each tab is an independent client.
 4. **Connect each tab:** in each tab, set a unique **Player Name** (e.g. `Player1` and `Player2`), choose a deck, and click **Connect & Ready**. The default host (`127.0.0.1`) and port (`8765`) match the bridge defaults.
 5. **Play the game** using the GUI buttons and card clicks. A **shot clock timer** in each player's stats panel shows the remaining time for the active priority holder before the server auto-passes or disconnects them.

Tip: you can mix GUI and terminal clients in the same game — for example, one player in the browser and one in the terminal.

Verbose mode (rubric prerequisite)

`--verbose` / `-v` on **either program** prints **every PDU sent and received**, labelled with direction, peer, timestamp, and the full JSON:

```
[14:03:22] SEND player_1 | {"type": "PRIORITY_GRANT", "player_id": "player_1", ...}
[14:03:23] RECV player_1 | {"type": "CAST_SPELL", "seq_num": 12, ...}
```

Without the flag both programs run quietly. This satisfies the “Verbose Mode Requirement” toggle in the project specification.

Debug flags (server)

- `--time-limit <ms>` — priority deadline (default 60000; RFC 4.2)
- `--seed <n>` — seed the RNG for reproducible shuffles/coin flips (demos)
- `--first {0,1}` — force which seat goes first (demos/tests)
- `--port <n>` — listen port

Tests

```
python server/tests/test_card_catalog.py # unit tests (catalog/decks)
python server/tests/test_game_state.py # unit tests (data model)
python server/tests/test_priority_stack.py # unit tests (tokens/stack)
python server/tests/test_effects.py # unit tests (effects/mana/SBAs)
python server/tests/test_integration_game.py # 10 protocol assertions
python server/tests/test_integration_combat.py # 3 combat assertions
python server/tests/test_integration_edge.py # 18 edge-case assertions
```

All are run from the project root. `test_integration_edge.py` derives a deterministic RNG seed by replicating the server’s seeded shuffle locally, so its scripted 10-turn game is fully reproducible.

Design summary

- **Transport (RFC 5):** every PDU is a 4-byte big-endian length prefix + UTF-8 JSON, max 65,535 bytes. `common.recv_pdu` reads exactly the framed bytes before parsing; malformed payloads yield `ERROR_INVALID_JSON`.
- **Threads:** the server runs one reader thread per client feeding a single event queue; the main thread runs the RFC Section 6 lifecycle (LOBBY → GAME_SETUP → MULLIGAN → IN_GAME → GAME_OVER → LOBBY , same TCP connections). PING is answered with PONG directly in the reader thread (independent heartbeat counter, RFC 5.4). The client runs reader, heartbeat (PING/30 s, 10 s PONG deadline) and stdin threads.
- **Authority (RFC 4.2/4.3):** all rules live in the server. The client never simulates outcomes; each personalized `GAME_STATE_UPDATE` (own hand only, opponent hand as a count) overwrites its view.
- **seq_num discipline (RFC 5.4):** the server keeps one monotonically increasing counter; a broadcast consumes one number. The client echoes the seq of the latest `PRIORITY_GRANT` for priority actions, the relevant request PDU for `MULLIGAN_CHOICE` / `DISCARD` (the `GAME_STATE_UPDATE`) and combat declarations (the `PHASE_TRANSITION`), and any last server seq for `CONCEDE`. Mismatches get `ERROR_STALE_ACTION` and, when the player still holds priority, a re-granted token. After an *illegal* (non-stale) action the server re-issues `PRIORITY_GRANT` with the **same** seq_num (RFC Sec. 11 item 3).
- **Rules implemented:** London mulligan; full phase sequence with first-turn draw skip; one land per turn at sorcery speed; implicit mana (declared `mana_payment` validated against untapped lands, which are then tapped); LIFO stack with priority windows and caster-retains-priority; fizzle on illegal targets at resolution; state-based actions after every event (lethal damage, 0 toughness, life ≤ 0, AP loses simultaneous-death ties); summoning sickness; combat with attack tapping, single-attacker blocks, multi-block damage ordering (`ASSIGN_DAMAGE_ORDER`), first-strike step when relevant, no trample; cleanup discard-to-7 loop; win by `LIFE_ZERO`, `DECK_EMPTY`, `CONCEDE`, `DISCONNECT` (incl. priority timeout); extra connections refused; lobby deck resubmission; duplicate ID rejection.
- **Card effects (≥ 5, RFC Appendix set):** Lightning Bolt / Shock (damage), Counterspell (counters a stack item), Giant Growth (+3/+3 until EOT), Healing Salve (lifegain), Divination (draw 2), Gray Merchant of Asphodel (ETB drain-2 optional **triggered ability** exercised through `TRIGGER_CHOICE` / `TRIGGER_CHOICE_RESPONSE`), Festering Imp (death trigger; two dying simultaneously exercises `TRIGGER_ORDER` / `TRIGGER_ORDER_RESPONSE`), Goblin Guide (haste), Youthful Knight (first strike), plus vanilla creatures and five basic lands.

Known deviations / interpretations

- A countered spell is announced with `STACK_RESOLVE result: "FIZZLE"` (the RFC only defines `RESOLVED` | `FIZZLE`,

with no dedicated “countered” result).

- Gray Merchant’s drain is modelled as an *optional* (“you may”) trigger so the `TRIGGER_CHOICE` flow of RFC 8.6.3 is exercised; declining discards it.
- `ACTIVATE_ABILITY` is answered with `ILLEGAL_ACTION` : the fixed card set defines no activated non-mana abilities, and mana is implicit (RFC 7.5). As a result the interactive client has no command that sends it (the server-side rejection path is still fully implemented).
- Summoning sickness is cleared for the active player’s creatures during their Cleanup (equivalent, for this card set, to “since your last turn began”).
- `--seed` / `--first` are non-RFC debug conveniences.

Work Distribution Matrix

Member 1 = John Lloyd Suñga · Member 2 = Daniel Rebudiao · Member 3 = Gaibril Gregorio · Member 4 = Carlo Barreo

Task / Feature	Member 1	Member 2	Member 3	Member 4
TCP Server: connection handling, framing, dispatch	<code>main.py</code> accept loop, lifecycle; <code>transport.py</code> <code>ClientConn</code> + dispatch table			
Game lifecycle: LOBBY, GAME_SETUP, MULLIGAN logic		<code>lobby.py</code> — <code>PLAYER_READY</code> , <code>LOBBY/SETUP</code> , London Mulligan	<code>card_catalog.py</code> — deck-list validation, <code>ILLEGAL_DECK</code>	
Turn & phase engine (all phases/steps, transitions)		<code>phase_engine.py</code> — phase/step machine, <code>PHASE_TRANSITION</code> , land drops		
Priority & Stack logic, spell/ability resolution			<code>priority.py</code> , <code>stack.py</code> , <code>effects.py</code> — <code>PRIORITY_GRANT</code> / <code>PASS</code> , <code>seq_num</code> tokens, LIFO stack, SBAs, triggers, card effects	
Combat system (attackers, blockers, damage)				<code>combat.py</code> — attackers/blockers, damage order, first strike
Client implementation & state rendering			<code>state_view.py</code> — personalized <code>GAME_STATE_UPDATE</code> views, hidden hands	<code>client/main.py</code> — full client & console rendering
PDU serialisation/deserialisation (all 25 PDU types)	<code>transport.py</code> — PDU codec, 4-byte length framing, <code>seq_num</code> -stamped send helpers			
Error handling, PING/PONG heartbeat, disconnect logic	<code>heartbeat.py</code> PING/PONG; error paths in <code>transport.py</code> / <code>main.py</code>		<code>game_state.py</code> — <code>GameOver</code> / <code>ClientGone</code> handling	
Verbose mode (client + server PDU logging, toggle on/off)	server-side verbose logging + <code>--verbose</code> flag (<code>transport.py</code> , <code>main.py</code>)			client-side <code>--verbose</code> PDU logging (<code>client/main.py</code>)
Testing & interoperability	✓	✓	✓	✓
README / documentation / AI disclosure	✓	✓	✓	✓

AI Usage Disclosure

1. Summary statement

AI assistance (Claude, ChatGPT, GitHub Copilot) was used during the development of this machine problem, primarily for extracting all requirements for the project, generating a task delegation plan for four developers that follow the Work Distribution Matrix, generating an initial implementation of the server package and client, drafting test scripts, and explaining other nuances in the specifications. All AI output was reviewed, tested, and modified by the group. Every member has read and can explain the modules attributed to them in the Work Distribution Matrix above, and the group jointly reviewed all remaining modules.

2. Tools used

AI tool	Version / model	Used by	Access (free / paid / school)	What it was used for
Claude	Opus 5, Fable 5	All members	Free, Paid	Extracting an overview of the project requirements,

				Generating an organized task delegation plan, Proper directory of the file architecture, Explanation of each task and how to implement it
ChatGPT	GPT-5	All members	Free	Explaining LIFO stack semantics, debugging framing bug
GitHub Copilot	Multi-model	All members	Free	Inline assisted autocompletion and guidance while writing files

3. Per-file AI involvement

Legend — **G** = AI generated the first draft, member reviewed/tested/edited · **A** = member wrote it, AI used only for debugging, refactoring, or explanation · **H** = hand-written, no AI involvement.

File	Level (See Legend)	Owner	What the AI did	What the member changed by hand
server/main.py	G	Suñga	Generating of pseudocode for needed functions, Help in unfamiliar processes, Argument parser help, Debugging	Writing of the functions
server/transport.py	G	Suñga	Generating logical flow for each needed functions, Help in PDU formatting when verbose is on, Explanation of unfamiliar error classes, Help in nuances of threading, Triple-checking of legal C->S PDU types	Writing of some functions, Improvements of bugged lines, Double-checking of legal C->S PDU types
server/heartbeat.py	A	Suñga	Debugging and clarifying understanding	Answering the ping
server/lobby.py	G	Rebudiao	Implemented the lobby, game setup, and mulligan lifecycle, including player validation, deck validation ready-state updates, disconnect handling, initial draws, first-player selection, and mulligan request handling.	Adjusted the implementation to match the RFC behavior, including deck replacement, request sequence validation, and seat reset handling.
server/phase_engine.py	G	Rebudiao	Implemented the turn and phase state machine, including untap, upkeep, draw, main phases, combat transitions, cleanup, land plays, and hand-size enforcement.	Updated the phase logic and validations to follow the RFC, especially priority handling, state broadcasts, request tokens, and turn transitions.
server/combat.py	G	Barreo	The AI generated the initial structure and logic for the combat system, including attackers, blockers, damage order, first strike, combat damage, and summoning sickness enforcement.	I fixed edge cases, corrected simultaneous damage calculation, and ensured the COMBAT_DAMAGE_RESULT broadcast matches RFC specifications.
server/card_catalog.py	G	Gregorio	Needed functions/methods for the file	Implementation of the methods / how to call them
server/game_state.py	A	Gregorio	The networking, Lobby, Combat, Stack resolution, Phase engine, Effects, Priority, Transport, and Client	Player state, Permanent management, Stack tracking, and Game-over/Disconnect Exceptions
server/state_view.py	A	Gregorio	The data formatting and cleaning the broadcast_state helper	I did everything
server/priority.py	A	Gregorio	Error-handling edge cases	I did everything
server/stack.py	G	Gregorio	The initial structure and logic, which handles the LIFO stack — spell casting, target validation, stack resolution with fizzle checks, state-based actions, and APNAP trigger ordering	you then fixed and debugged the edge cases like mana payment validation, sorcery-speed timing checks, and the trigger choice/order flows
server/effects.py	G	Gregorio	Debugged Edge Cases like	I did all of the card effect

			generic mana coverage checks and the countered-spell graveyard handling	logic
client/main.py	A	Barreo	The AI helped with boilerplate code for client-server connection, PDU sending/receiving, and verbose mode setup.	I wrote the client implementation, including handling all PDU types, rendering authoritative GAME_STATE_UPDATE, discarding local state, and error handling without crashing.
shared/cards.json , decks/*.txt	A	All Members	Double-checked correctness of necessary values	We wrote everything
README.md	G	all	General formatting of the whole README file	Fixed the TODO markings and grammar.

4. Member attestation

Member	Modules I can explain line-by-line	AI tools I personally used
John Lloyd Suñga	main.py , transport.py , heartbeat.py	Claude, GitHub Copilot
Daniel Rebudiao	lobby.py , phase_engine.py	Claude, Big Pickle
Gaibril Gregorio	card_catalog.py , game_state.py , state_view.py , priority.py , stack.py , effects.py	Claude, ChatGPT
Carlo Barreo	combat.py , client/main.py	DeepSeek